ETH
Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

Semester Project Proposal

νActionGUI: Process Permissions & More Data Subject Rights

Supervisor: Dr. Srđan Krstić, Hoang Nguyen
Professor: Prof. David Basin

1. Prerequisites

Some familiarity with databases, OCL, and model-driven development.[1]

Some familiarity with the General Data Protection Regulation (GDPR) [4], particularly Chapter 3, Rights of the Data Subject.

2. Motivation

Enforcing security (e.g., fine-grained access control [7]) and privacy (e.g., purpose limitation [4]) policies in today's data-centric applications is both critical and challenging. It is critical because of the high economic and social consequences of enforcement failure. It is challenging as it is not yet well-supported by the current technologies: it is implemented in an ad-hoc manner, which makes its maintenance and evolution error-prone and time consuming.

In particular, developers must address the policies in a consistent and cross-cutting manner. As a result, broken access control is the most common web application vulnerability according to the OWASP Top 10 report [6], whereas data protection violation fines have reached an all time high [3].

Previously, Basin et al. [2] have proposed a model-driven security (MDS) approach for building secure-by-design systems. In addition to the application, they model security policies expressed as fine-grained access control (FGAC) permissions. The idea is that software engineer first creates a design model in a design modeling language (e.g., a UML class diagram), which represents the system's state-space. A security engineer then creates a security model in a security modeling language, which defines the system's FGAC permissions. These permissions consist of the (static) role-based access control specification and the (dynamic) OCL constraints. Using model transformations, a enforcement code is generated from the two models.

ActionGUI [1] is a tool that supports the above MDS methodology and generates fully functional web applications with a configured enforcement mechanism for the FGAC policies specified by the security model.

Recently, Krstić et.al. [5] extended MDS to include privacy support by proposing a new model that captures two common privacy requirements: *purpose-limitation* and *consent*. They also developed a new tool, νActionGUI, which supports FGAC, like in ActionGUI [1], and also the aforementioned *purpose-limitation* and *consent* privacy requriements. Finally, they demonstrated the tool usage by implementing three simple case study web applications [5].

[1] E.g., see the course on Security Engineering: `https://infsec.ethz.ch/education/as2022/seceng.html`

3. Objective

The objective of this semester project is to further improve the implementation of Krstić et al. [5] in multiple dimensions. First, we aim to extend the security modeling language with method permissions, field constraints, and symmetrical constraints. Second, we aim to extend the privacy model for better GDPR compliance, including key rights of the data subject from Chapter 3 of the GDPR. Finally, we will develop supporting tools for syntax highlighting, validation, and evaluate the extended version of the tool via a real-world case study.

4. Tasks

In what follows, we split the tasks into three work packages:

4.1. Improving Security Modeling Language and Semantics

1. Study the existing model-to-code transformation in νActionGUI.
2. Add support for symmetrical constraints (by default) to reduce workload for security engineers. Option to write asymmetrical constraints should still be available.
   Whenever possible, automatically infer symmetrical constraints for actions on attributes or association-ends that are linked via an association.
3. Add support for specifying method execution permissions.
   Context. The data model in νActionGUI now includes method definitions.
   - Extend the security modeling language to allow specifying permissions for method executions. As these correspond to process permissions they may be in conflict with the existing data permissions. The tool should provide an option for process permissions to override data permission within the corresponding method.
   - Update the model-to-code transformation function to support these permissions.
4. Add explicit support for constraints on field initialization.
   Context. Currently, object creation initializes all attributes with default values (e.g., null, 0, or $\{\}$) and there is no distinction between a property initialization and its subsequent update.
   - Extend the security modeling language to allow permissions for object creation with specific parameters. These permissions may depend on the argument objects.
   - Extend the property actions with the new init and write actions. The first action presents the initialization of the property, while the second presents an update of that property after it has been initialized.
   - Statically cross-validate these new object creation permissions against the data model.

4.2. Improving the Privacy model Transformation

5. Extend the model transformation to support GDPR compliance, including
   - Purpose limitation with legal entities (Article 5(b), Chapter 2).
   - Right of access by the data subject (Article 15, Section 2, Chapter 3).
   - Right to rectification (Article 16, Section 3, Chapter 3).
   - Right to erasure (Article 17, Section 3, Chapter 3).
   - Right to restriction of processing (Article 18, Section 3, Chapter 3).
   - Right to object (Article 21, Section 3, Chapter 3).

   Generate additional artifacts and functionalities that enable users to review and manage their personal data and their rights directly.
6. Ensure that the security model aligns with these privacy requirements.
   Adapt the security enforcement mechanisms to ensure that privacy rights are respected (e.g., legal entities accessing personal information with proper cause, users managing their personal data).

4.3. Syntax Highlighting and Evaluation

7. (Optional) Create a VSCode client and a language server to highlight and validate the models, respectively.
   Provide support for syntax highlighting and live model validation with νActionGUI.
8. (Optional) Extend the functionality of the language server to provide semantics-based feedback (e.g., constraint simplification, unsatisfiability, etc.).
   Extend the aforementioned validation with new validation rules from the previous work packages.
9. Verify the soundness of the extensions by (property) testing it.
10. Develop a case study.
    - Use the updated tool to develop the Event Platform web application.
    - Conduct a comparison with the current version, for development effort and usability.

5. Deliverables

The following deliverables are due at the end of the project:

Source code Complete development including easily reproducible evaluation.

Presentation At the end of the project, a 20 minute presentation must be given in an InfSec group seminar. It should give an overview and discuss the most important highlights of the work.

References

[1] David Basin, Manuel Clavel, Marina Egea, Miguel Angel García de Dios, and Carolina Dania. A Model-Driven Methodology for Developing Secure Data-Management Applications. *IEEE Trans. Software Eng.*, 40(4):324–337, 2014.

[2] David Basin, Jürgen Doser, and Torsten Lodderstedt. Model driven security: from uml models to access control infrastructures. *ACM Transactions on Software Engineering and Methodology*, 15(1):39–91, January 2006.

[3] CMS. Gdpr enforcement tracker. `https://www.enforcementtracker.com/`, 2024.

[4] European Parliament and Council. Regulation (EU) 2016/679, 2016. `https://eur-lex.europa.eu/eli/reg/2016/679/oj`.

[5] Srdan Krstic, Hoang Nguyen, and David A. Basin. Model-driven privacy. *Proc. Priv. Enhancing Technol.*, 2024(1):314–329, 2024.

[6] Open Worldwide Application Security Project. OWASP Top 10. `https://owasp.org/Top10/2025/`, 2025.

[7] Ravi S Sandhu and Pierangela Samarati. Access control: principle and practice. *IEEE communications magazine*, 32(9):40–48, 1994.

A. Action hierarchy revisited

Let $R = \{\text{entity}, \text{association}, \text{attribute}, \text{end}, \text{method}\}$ be a set of resources and let $\leq_R = \{(\text{attribute}, \text{entity}), (\text{end}, \text{entity}), (\text{method}, \text{entity}), (\text{end}, \text{association})\}$ define a subresource relation. An entity resource has attributes, ends, and methods as subresources, whereas association has ends.

Let $\text{Act} = \{\texttt{read}, \texttt{init}, \texttt{update}, \texttt{write}, \texttt{add}, \texttt{remove}, \texttt{execute}, \texttt{execute}^*, \texttt{fullAccess}, \texttt{create}, \texttt{delete}\}$ be a set of action names. Some action names name composite actions that consist of other actions. The following relation defines this relationship on action names:

$\leq_c = \{(\texttt{read}, \texttt{fullAccess}), (\texttt{write}, \texttt{fullAccess}), (\texttt{create}, \texttt{fullAccess}), (\texttt{delete}, \texttt{fullAccess}), (\texttt{execute}, \texttt{fullAccess}), (\texttt{init}, \texttt{write}), (\texttt{update}, \texttt{write})\} \subset \text{Act}^2$

Let $AA = \{(\text{entity}, \texttt{create}), (\text{entity}, \texttt{delete}), (\text{method}, \texttt{execute}), (\text{method}, \texttt{execute}^*), (\text{attribute}, \texttt{read}), (\text{end}, \texttt{read}), (\text{attribute}, \texttt{init}), (\text{attribute}, \texttt{update}), (\text{end}, \texttt{add}), (\text{end}, \texttt{remove})\} \subset R \times \text{Act}$ be the set of atomic actions.

We define an action hierarchy $\leq_A \subseteq (R \times \text{Act})^2$ as

$$(r, a) \leq_A (r', a') \iff (r \leq_R r' \wedge a = a') \vee (r = r' \wedge a \leq_c a')$$

The final set of actions A supported by nuActionGUI is defined inductively as follows:

$$(r, a) \in A \iff (r, a) \in AA \vee \exists (r', a') \in A.\ (r', a') \leq_A (r, a)$$

TODO: special consistency for: add, remove, (init), update, write to associations and ends

TODO: Do we need init and update versions of add and remove? Can we make this orthogonal?